



PROJECT REPORT

Relativistic Schwarzschild Black Hole Renderer

Real-time GPU ray-bending visualisation of a non-rotating
black hole and its glowing accretion disk

Java 17 · LWJGL 3 · OpenGL 3.3 Core · GLSL

NAME	Dhruv Sagar Inamdar
PRN	1262240143
ROLL NUMBER	24
BRANCH	TYCSE - A

1. What this project is about

This is an interactive, real-time visualisation of a **non-rotating (Schwarzschild) black hole** and its glowing accretion disk. It recreates the recognisable black-hole look: a dark event-horizon shadow, lensed arcs of hot plasma, a bright photon-ring enhancement, distorted background stars, and a brighter approaching side of the disk.

It is a renderer rather than a full astrophysics simulator. The project uses physics-inspired Schwarzschild ray bending and radiative-transfer approximations to make the image convincing at interactive frame rates. The default scene represents a 10-solar-mass black hole, viewed from a configurable orbital camera.

2. Project Information

Project stack

Layer	Implementation
Language and build	Java 17 with Gradle
Native/window layer	LWJGL 3 with GLFW
Graphics API	OpenGL 3.3 Core
Per-pixel renderer	GLSL fragment shaders
Tests	JUnit 5

Rendering flow

```
Java app + camera/input/config
    |
    v
Fullscreen-triangle GPU draw
    |
    v
GLSL: RK4-inspired bent-ray march + procedural accretion disk
    |
    v
HDR scene framebuffer
    |
    v
bright-pass -> horizontal blur -> vertical blur
    |
    v
ACES tone mapping + gamma -> screen, with optional HUD
```

For each screen pixel, the main fragment shader starts a ray at the camera and marches it backward through the scene. It evolves the ray with a fourth-order Runge-Kutta (RK4) step based on a Schwarzschild-inspired acceleration term. Rays that reach the event horizon are black; escaping rays can see the procedural starfield. While travelling through the disk volume, the shader samples noisy plasma density, thermal colour, absorption, and emission. It then applies approximate Doppler/gravitational brightness effects and Beer-Lambert transmittance.

The Java renderer sends the camera basis, black-hole radii, disk settings, selected palette, debug mode, and quality-preset values to the shader as uniforms. It also runs the post-processing passes and renders the optional telemetry/palette UI.

Interactive and verification features

- Mouse drag or **A/D** orbits; arrow keys pitch; **Q/E** rolls; **W/S** zooms.
- **1/2/3** select low/medium/high quality. The presets use 96/160/240 maximum geodesic steps respectively.
- **Tab** cycles diagnostic views for density, temperature, optical depth, lensing, and step count.
- **B** toggles bloom, **C** opens the palette picker, and **F12** saves a PNG capture.
- `--capture-benchmark` renders an automated visual-verification set and measures GPU frame time with OpenGL timer queries.

3. Motive

Because black holes are already doing the most, and rendering one as a flat black circle would be criminally underwhelming.

The project is a practical way to connect Java OOP, native graphics programming, numerical methods, and physics in one visual result. It deliberately avoids a game engine so the interesting machinery stays visible: create the OpenGL window, compile shaders, make framebuffers, pass data to the GPU, bend rays, then polish the glow.

4. How Java is implemented in this project

Java is the **application and orchestration layer**. The final colour of each pixel is calculated in GLSL on the GPU, but Java builds and controls the whole system around it.

Java area	What it does
<code>cosmic.app.BlackHoleApp</code>	Entry point; creates the scene, runs the frame loop, handles controls, screenshots, and the benchmark/capture mode.
<code>cosmic.app.Window</code>	Wraps GLFW initialisation, the native OpenGL window/context, resize callbacks, event polling, and cleanup.
<code>cosmic.graphics.Renderer</code>	Owns shader programs, a fullscreen triangle, a 3D noise texture, HDR/blur framebuffers, the render-pass sequence, and HUD.
<code>cosmic.graphics.Camera</code>	Computes orbital-camera position and forward/right/up vectors for shader ray generation.
<code>cosmic.model.BlackHole / AccretionDisk</code>	Hold domain values such as mass, Schwarzschild/photon-sphere scales, disk radii, temperature, density, and emission settings.
<code>cosmic.physics</code>	Provides readable CPU-side Schwarzschild/geodesic/RK4 classes and supports the project's physics model and tests. The live image path is the GLSL raymarch.
<code>cosmic.util.Config</code>	Loads <code>blackhole.properties</code> , giving editable startup values without recompiling.
<code>cosmic.util.ResourceLoader</code>	Loads classpath resources and recursively resolves GLSL <code>#include</code> files.
Vec3, enums, interfaces, exceptions	Demonstrate Java records/value types, encapsulation, enums for quality/palettes, the <code>Resettable</code> contract, and purpose-specific error handling.

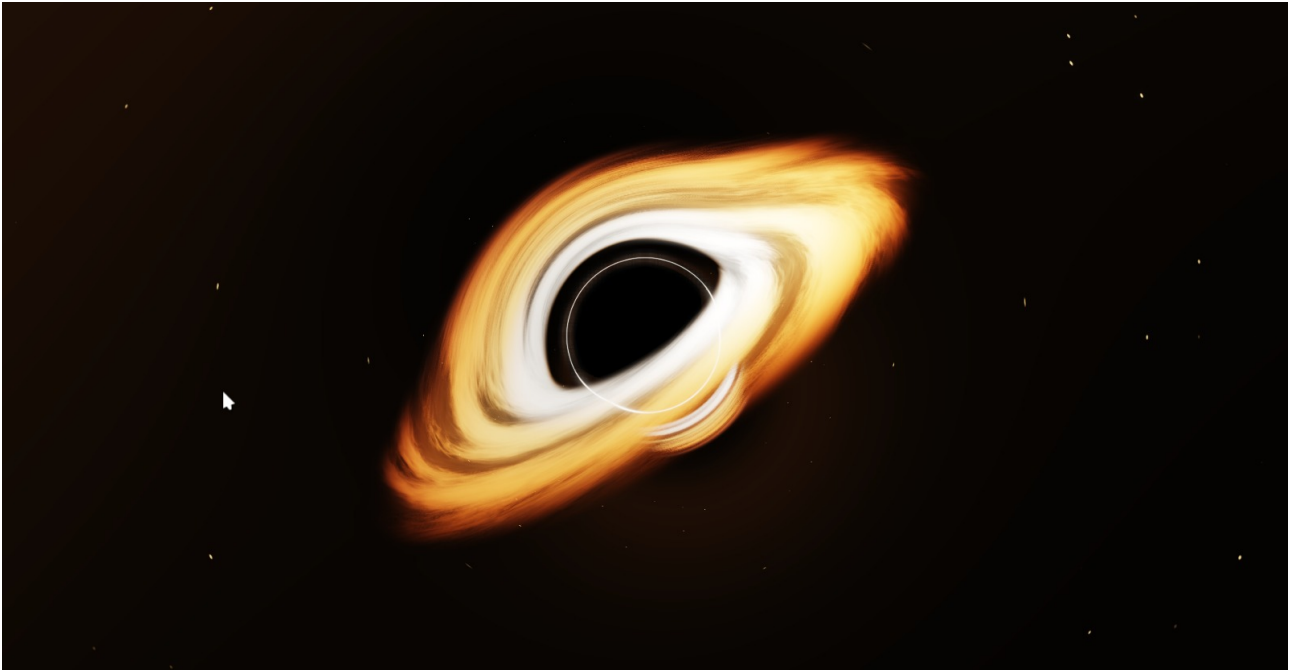
The Gradle build targets Java 17 and pulls LWJGL 3.3.3 (core, GLFW, OpenGL, STB) plus Linux native libraries. The included JUnit tests passed with `./gradlew test`.

Important implementation note

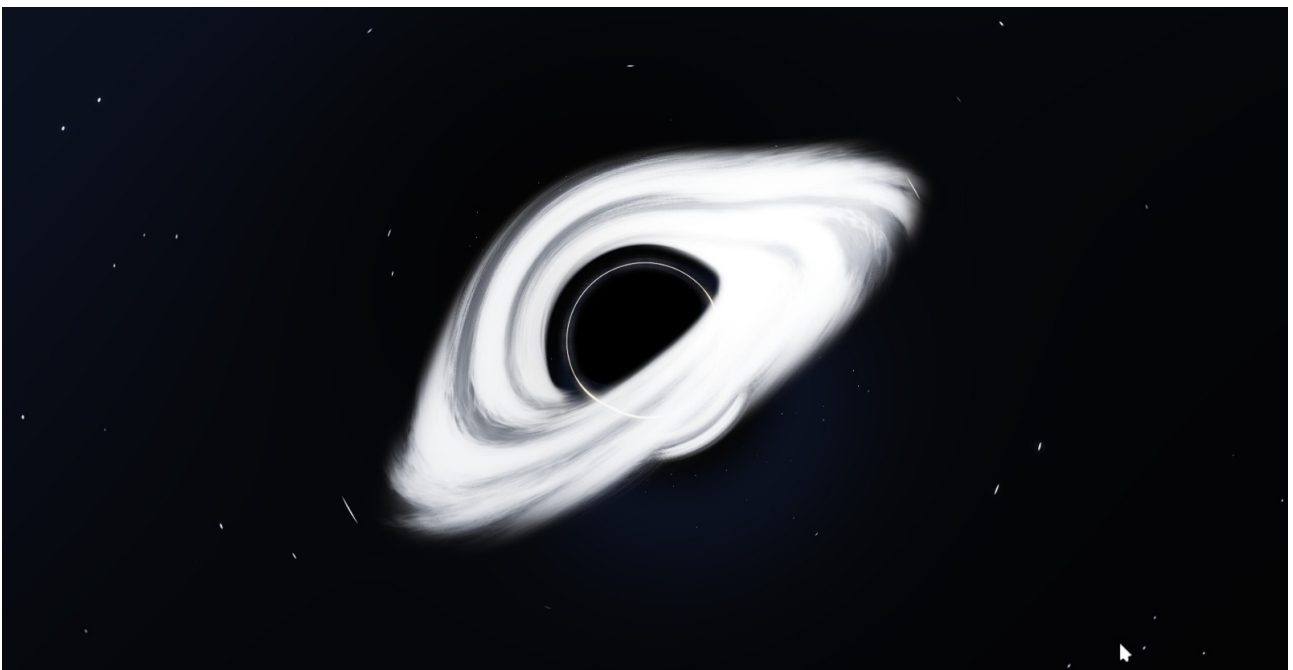
The CPU `RayIntegrator`, `Geodesic`, and some Java colour/model helpers are useful domain code, but they do **not** generate the real-time pixels. `shaders/blackhole.frag` is the source of truth for the displayed ray integration, disk medium, and lighting. Java passes the scene state to that shader and displays its output.

5. Screenshots

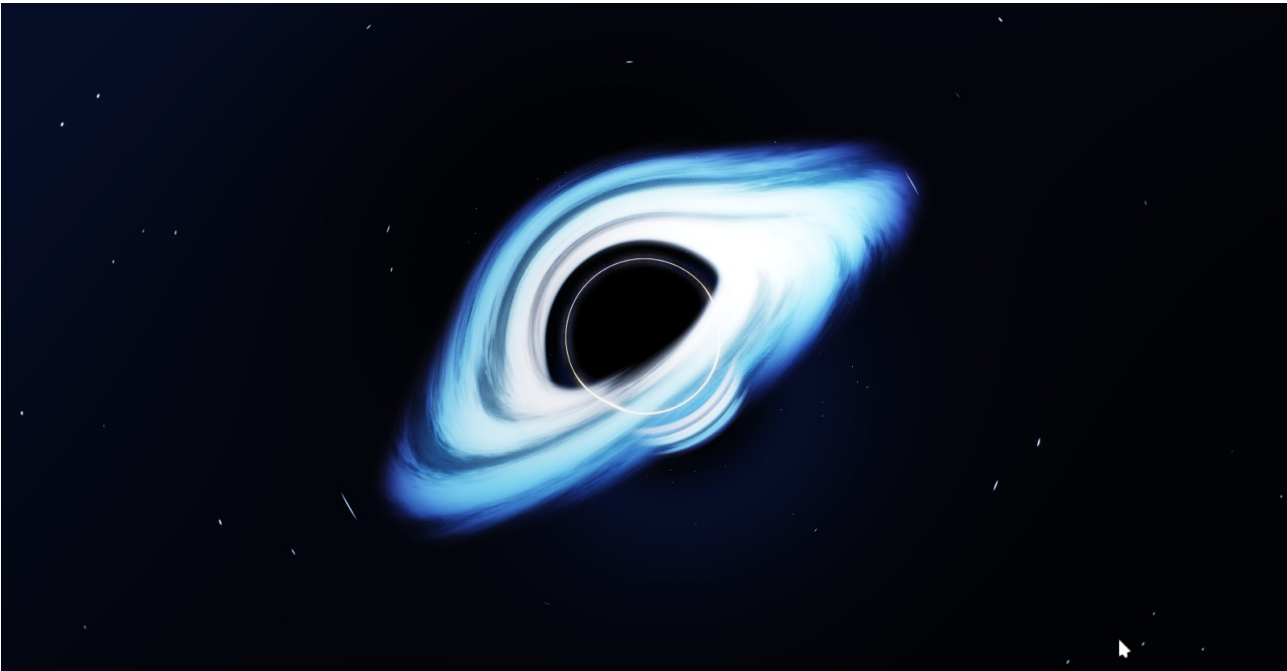
The following screenshots show different cinematic views of the lensed accretion disk, including the warped far-side arc and asymmetric hot regions, rendered in different colour palettes.



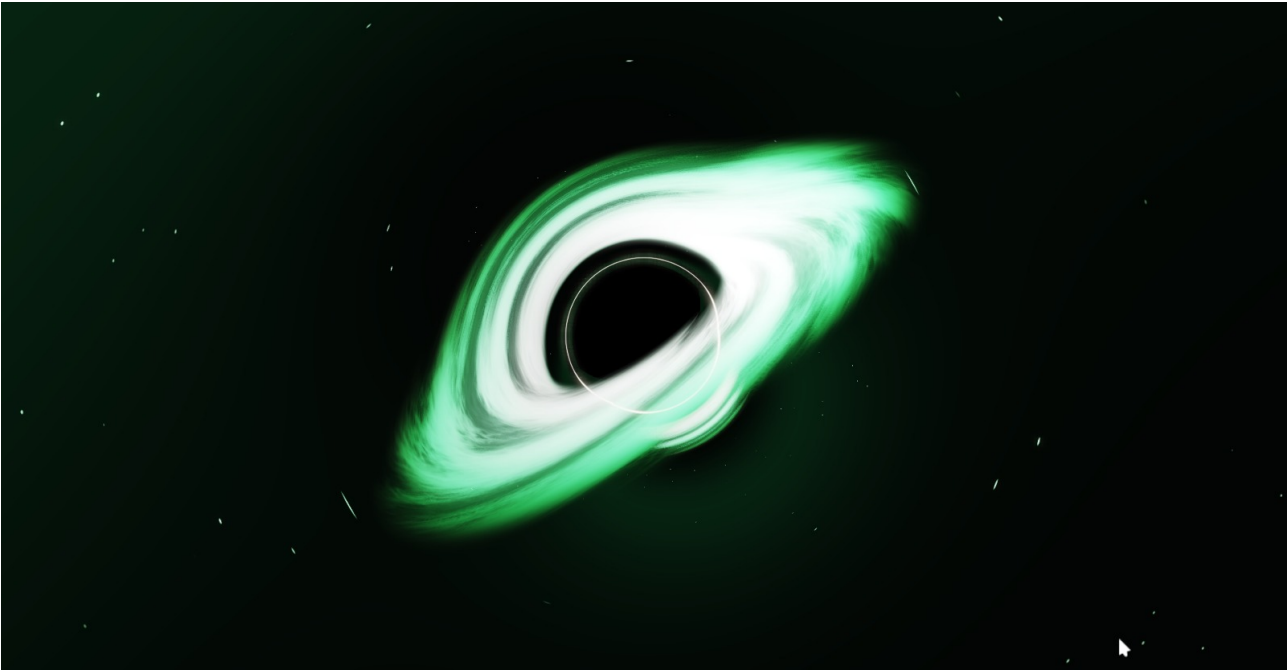
Screenshot 1 — Classic amber palette



Screenshot 2 — Monochrome white palette



Screenshot 3 — Blue palette



Screenshot 4 — Green palette



Screenshot 5 — Violet palette

Run it

```
./gradlew run
```

To produce the automated benchmark and debug-view captures:

```
./gradlew run --args="--capture-benchmark"
```